

Robot Tuning Guide

This guide reflects the current firmware in [MazeMap.ino](#). The main tuning constants live in the `Config`, `DiagnosticConfig`, and `LedCalibrationConfig` blocks there. Sensor placement and default wall-sensor models live in [Vehicle.cpp](#).

Where To Tune

- Edit main mission and control constants in [MazeMap.ino](#).
- Edit wall sensor physical positions only if the hardware moved in [Vehicle.cpp](#).
- Use diagnostic and maneuver-test logs to decide changes before raising speed.

Boot Modes

The firmware checks boot straps in this order:

1. `38-39`: wall sensor LED calibration
2. `29-30`: maneuver file test
3. `30-31`: primary diagnostic battery
4. no recognized strap: normal mission

If more than one strap is installed, the first match above wins.

1. Wall Sensor LED Calibration Mode

- Strap: short pins `38` and `39` before reset or power-up.
- Purpose: bench-only emitter timing check for the wall sensor LEDs.
- What it does:
 - Starts with the front LEDs toggling and the side LEDs off.
 - Prints the front LED square-wave frequency over serial.
 - Waits for you to remove the `38-39` jumper.
 - After the jumper is removed, switches to side LED toggling and keeps running.
- What the robot should be like when turned on:
 - Stationary on the bench.
 - Wheels clear of the floor, or otherwise unable to drive into anything.
 - Access to a scope, photodiode, or light sensor if you are checking emitter timing or brightness.
- Useful details:
 - Front LED half-period is based on the front sensor settle time and is currently `60 us`, so the square wave is about `8.33 kHz`.
 - Side LED half-period is currently `10 us`, so the square wave is about `50 kHz`.
 - This mode does not drive the robot.

2. Maneuver File Test Mode

- Strap: short pins `29` and `30` before reset or power-up.
- Purpose: repeatable motion tuning from an SD-card script.
- What it does:
 - Runs the same startup initialization as normal mission mode.
 - Runs startup wall calibration.
 - Loads `test.txt` from the SD card.
 - Executes the maneuver queue.
 - Writes `maneuver_test.csv`.
- What the robot should be like when turned on:
 - Physically placed at the start pose and facing `Up`.

- In a startup calibration pocket where the robot can gently touch the west, east, and south walls during startup.
- On a floor area that has enough clear space for every maneuver in `test.txt`.
- With a readable SD card inserted.
- `test.txt` format:
 - Tokens can be separated by commas, spaces, tabs, or semicolons.
 - `#` and `//` comments are ignored.
- Use this mode when:
 - Straight motion is good enough to leave the bench.
 - You want repeatable turn or arc tuning without running a full maze.
 - You want wall-calibration metadata and motion telemetry in one file.

3. Primary Diagnostic Battery Mode

- Strap: short pins `30` and `31` before reset or power-up.
- Purpose: open-floor logging for wheel, IMU, and controller tuning.
- What it does:
 - Does not run startup wall calibration.
 - Assumes the current pose is the start pose, facing `Up`.
 - Logs a fixed test sequence:
 - startup settle
 - baseline idle
 - repeated 90 degree and 180 degree turns
 - short straight out-and-back
 - long straight out-and-back
 - clockwise square loop
 - counterclockwise square loop
 - final idle
 - Writes `diagNNN.csv` on Teensy, or `diagnostic_log.csv` off target.
- What the robot should be like when turned on:
 - Stationary on a flat open floor.
 - Facing the intended `Up` direction.
 - Centered inside a clear square at least `0.68 m x 0.68 m`, because the boundary half-span is `0.34 m`.
 - Not between maze walls; this mode is for open-floor tests.
 - With a readable SD card inserted.
- Use this mode when:
 - Tuning wheel feedforward and wheel velocity PI.
 - Checking gyro bias stability and turn response.
 - Verifying acceleration, braking, and basic heading control before wall following.

4. Normal Mission Mode

- Strap: none.
- Purpose: full maze operation.
- What it does:
 - Runs startup wall calibration.
 - Observes the start cell.
 - Explores the maze.
 - Returns to start.
 - Runs speed run 1.
 - Waits for an inter-run service jumper.

- Repeats startup wall calibration.
- Runs speed run 2.
- What the robot should be like when turned on:
 - Physically placed at the start pose and facing Up.
 - In a startup calibration pocket where west, east, and south wall touch-off is valid and square.
 - On a maze/floor setup that lets the robot leave that pocket and begin exploration after calibration.
- Important note:
 - The startup calibration is stricter than "just put the robot somewhere near the start." It explicitly touches west, east, and south walls and then assumes those contacts correspond to one-cell geometry.

Not A Boot Mode: Inter-Run Service

During normal mission mode, after the first speed run the firmware stops and waits for a service jumper:

- Service jumper: short pins 34 and 35.
- Intended use: tire cleaning or other between-run service.
- Operator workflow:
 - Install the 34-35 jumper before lifting the robot.
 - Place the robot back at the start facing Up.
 - Remove the jumper.
 - The firmware then reruns startup wall calibration and begins speed run 2.

Recommended Tuning Order

1. Fix Geometry And Direction First

Do this before touching any gains:

- Verify motor polarity with `kInvertLeftMotor` and `kInvertRightMotor`.
- Verify encoder polarity with `kInvertLeftEncoder` and `kInvertRightEncoder`.
- Set `kEncoderCountsPerRev` only to the real hardware value.
- Set `kWheelDiameterM` from measured distance error.
- Set `kTrackWidthM` from yaw error during differential motion.
- If the wall sensors physically moved, update their positions in [Vehicle.cpp](#).

Do not use encoder count or polarity constants to hide wheel diameter or track-width mistakes.

2. Validate The Wall Sensors On The Bench

Use LED calibration mode first.

- Confirm all emitters are switching.
- Confirm the front pair and side pair are wired to the correct channels.
- Check that ambient and lit readings separate cleanly.
- If one channel is noisy or saturated on the bench, fix optics, wiring, or analog gain before tuning control loops.

3. Make Startup Wall Calibration Repeatable

Use normal mission mode or maneuver file test mode in the startup calibration pocket.

Tune these groups until startup is gentle and repeatable:

- `kStartupWallCalibrationSettleMs`
- `kStartupWallCalibrationSpeedMps`
- `kStartupWallCalibrationAccelMps2`
- `kStartupWallCalibrationDecelMps2`
- `kStartupWallCalibrationTurnMaxOmegaRadps`

- `kStartupWallCalibrationTurnAccelRadps2`
- `kWallTouchContactStandoffM`
- `kWallTouchDriveCommand`
- `kWallTouchHeadingDrivePerRad`
- `kWallTouchYawD`
- `kWallTouchContactSpeedThresholdMps`
- `kWallTouchMinApproachDistanceM`
- `kWallTouchMaxApproachDistanceM`
- `kWallTouchConfirmSamples`

For front-sensor curve quality, adjust:

- `kStartupWallCalibrationFrontFarOffsetM`
- `kStartupWallCalibrationFrontStepM`
- `kStartupWallCalibrationFrontPointCount`

What good startup calibration looks like:

- The robot touches walls gently, without bouncing or yawing away.
- Repeated startups produce similar wall-calibration data.
- In `maneuver_test.csv`, `estimated_touch_standoff_m` is close to `configured_touch_standoff_m`.
- The robot returns to the center cleanly after the calibration routine.

4. Tune Wheel Loops In Diagnostic Mode

Run the primary diagnostic battery on open floor and use the CSV.

Tune in this order:

1. `kWheelStaticFeedforward`
2. `kWheelVelocityFeedforward`
3. `kWheelVelocityKp`
4. `kWheelVelocityKi`
5. `kWheelIntegralLimit`

What to inspect in the log:

- `cmd_linear_mps` versus `left_velocity_mps` and `right_velocity_mps`
- `left_drive_cmd` and `right_drive_cmd`
- whether low-speed commands break stiction cleanly
- whether cruise speed needs large steady-state error to hold target
- whether braking and restart cause slow recovery from integrator windup

Typical symptoms:

- Robot twitches around zero: lower `kWheelStaticFeedforward`.
- Robot will not start moving at low command: raise `kWheelStaticFeedforward`.
- Cruise speed is low even with modest feedback action: raise `kWheelVelocityFeedforward`.
- Speed oscillates or chatters: lower `kWheelVelocityKp`.
- Long slow surging after transients: lower `kWheelVelocityKi`.

5. Tune Gyro Bias And Turn Control

Still in diagnostic mode:

- Check `baseline_idle` and `final_idle` first.
- If stationary `gyro_raw_radps` wanders too much between startups, increase `kGyroBiasSamples`.
- If startup time matters and bias is already stable, reduce `kGyroBiasSamples`.

Then tune:

- `kTurnHeadingKp`
- `kTurnYawD`
- `kAngleToleranceRad`
- `kAngularSpeedToleranceRadps`

What good turns look like:

- 90 degree and 180 degree phases stop without ringing.
- Overshoot is small and repeatable.
- The robot does not scrub badly or hunt around the target angle.

6. Tune Straight Heading And Wall Following

Once open-floor motion is stable, move into a corridor or other wall test fixture.

Tune the sensing and wall-following group:

- `kExpectedSideWallDistanceM`
- `kFrontWallOnThresholdM`
- `kFrontWallOffThresholdM`
- `kSideWallOnThresholdM`
- `kSideWallOffThresholdM`
- `kStraightHeadingKp`
- `kStraightYawD`
- `kWallCenterGain`
- `kFrontSkewGain`

Use `maneuver_test.csv` when you need wall telemetry during controlled repeated motion.

What to inspect:

- `ws_*_distance_m`
- `front_wall`, `left_wall`, `right_wall`
- `corridor_error_m`
- `front_skew_m`

Typical symptoms:

- Robot rides one wall while appearing straight: adjust `kExpectedSideWallDistanceM`.
- Walls latch too late: raise the corresponding `On` threshold.
- Walls stay latched after an opening: lower the corresponding `Off` threshold.
- Corridor following hunts left-right: lower `kWallCenterGain`.
- Robot finishes straight segments angled into front walls: raise `kFrontSkewGain` carefully.

7. Tune Arc Tracking With Maneuver File Tests

After straight and turn behavior are solid, use `test.txt` to repeat the same maneuvers.

Tune:

- `kArcHeadingKp`
- `kArcYawD`
- `kDistanceToleranceM`
- `kSpeedToleranceMps`

This is the right place to tune diagonal and arc behavior because the exact same script can be rerun after each change.

8. Raise Exploration And Speed-Run Aggression Last

Only after the robot is already reliable:

- Tune search behavior with:

- `kSearchMaxSpeedMps`
- `kSearchAccelMps2`
- `kSearchDecelMps2`
- `kSearchLateralAccelMps2`
- `kSearchTurnMaxOmegaRadps`
- `kSearchTurnAccelRadps2`
- Tune final racing conservatively with:
 - `kSpeedRunScale`
 - `kRacingFanDutyCycle`

Recommended rule:

- If the robot is not boringly reliable in search, do not raise `kSpeedRunScale`.

Logs To Use

Diagnostic Mode

- File: `diagNNN.csv`
- Best for:
 - wheel feedforward and PI
 - gyro bias quality
 - open-floor turn tuning
 - acceleration and braking checks

Maneuver File Test Mode

- File: `maneuver_test.csv`
- Best for:
 - wall-calibration metadata
 - repeated turn and arc tests
 - corridor-following checks with controlled paths
 - validating `test.txt` sequences before a full mission

Normal Mission Mode

- Best for final validation, not first-pass tuning.
- Use it only after the lower-risk modes are stable.

Practical Rules

- Change one parameter group at a time.
- Re-run the same mode after each change.
- If startup wall calibration is inconsistent, stop there and fix that before touching search or speed-run gains.
- If open-floor diagnostics are unstable, do not move on to corridor or maze tuning.
- If maneuver tests are inconsistent, do not raise mission speed.